

拓扑 AC | CSP-S 线上联测

TopoAC Algorithm Contest

第二轮 第三场

题目名称	跳跃表	幸运评级	打麻将	树上守望
题目类型	传统型	传统型	传统型	传统型
目录	jump	rating	mahjong	watch
可执行文件名	jump	rating	mahjong	watch
输入文件名	jump.in	rating.in	mahjong.in	watch.in
输出文件名	jump.out	rating.out	mahjong.out	watch.out
每个测试点时限	2.0 秒	2.0 秒	4.0 秒	2.0 秒
内存限制	512 MiB	512 MiB	512 MiB	256 MiB
测试点数目	25	25	25	25
子任务数目	5	3	4	6
测试点是否等分	否	否	否	否

提交源程序文件名

对于 C++ 语言	jump.cpp	rating.cpp	mahjong.cpp	watch.cpp
-----------	----------	------------	-------------	-----------

编译选项

对于 C++ 语言	-O2 -std=c++17 -Wall
-----------	----------------------

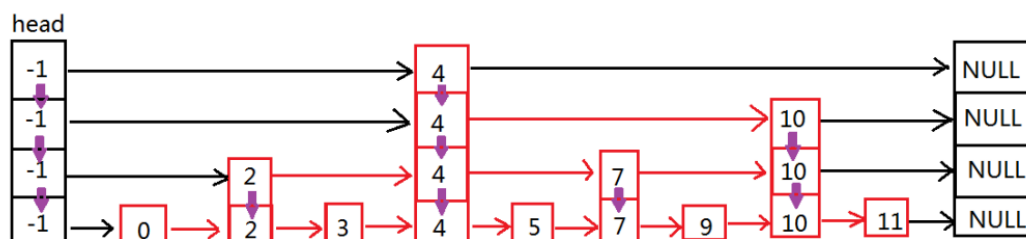
注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 `0`。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 禁止在源代码中改变编译器参数（如使用 `#pragma` 命令），禁止使用系统结构相关指令（如内联汇编）和其他可能造成不公平的方法。
7. 请注意所有题目均使用子任务计分，只有通过属于该子任务的全部数据后，才能获得该子任务的分数。

跳跃表 (jump)

【题目背景】

跳跃表的结构是这样的：



http://blog.csdn.net/LF_2016

它本质上是一个链表，有若干层。

最下层的链表维护了所有的元素。对于每一个元素，随机了一个 $[1, k]$ 的层数，来表示这个节点需要维护几层。

例如上图中的值为4的节点，维护了4层。

然后，把每一层的节点，按照从左到右的顺序连接起来。

这样的好处是什么呢？

(1) 在这个跳跃表上，可以轻松完成二分一个元素位置的事情。我们只需要从最高的那一层开始，要么向右跳，要么向下跳即可。比如，我们现在要查询数字9所在的位置，只需要沿着（左边是数字，右边是层数）

$(-1, 4), (4, 4), (4, 3), (4, 2), (7, 2), (7, 1), (9, 1)$ 的顺序查找到9即可。

(2) 也可以轻松的完成插入元素的工作。我们只需要二分找到其出现的位置，然后给它随机一个层数，再把每一层对应的前置节点连接起来即可。

(3) 容易感性理解，这个结构单次插入或查询的期望时间复杂度均为 $O(\log n)$ 。

【题目描述】

有 N 个按编号 $1, 2, \dots, N$ 排好序的元素。你要建立一个恰有 K 层的跳跃表。

表头 head 出现在所有层。对每个元素 i ，你可以选择一个高度 h_i ，满足 $1 \leq h_i \leq K$ ；元素 i 出现在第 1 层到第 h_i 层。每一层中的节点均按编号从小到大连接。特别地，第 1 层包含全部元素；某些较高层可以没有元素。

查询元素 i 时，指针从第 K 层的 head 开始，并重复以下过程：

1. 如果当前层中，当前节点右侧的下一个节点编号不超过 i ，则向右移动到该节点；
2. 否则，若当前层不是第 1 层，则向下一层移动；
3. 指针到达元素 i 时，查询结束。

一次查询的代价只计算指针向右移动的次数，向下移动不计代价。

元素 i 会被查询 x_i 次。请为所有元素选择高度，使总查询代价

$$\sum_{i=1}^N x_i f(i)$$

最小，其中 $f(i)$ 表示查询元素 i 时向右移动的次数。输出这个最小值。

【输入格式】

从 *jump.in* 读入数据。

第一行两个整数 N, K 。

第二行 N 个整数 $x_1, x_2, \dots, x_N$ ，表示各元素的查询次数。

【输出格式】

输出到 *jump.out* 文件。

输出一个整数，表示最小总查询代价。

【样例 1 输入】

```
1 3 2
2 5 2 5
```

【样例 1 输出】

```
1 14
```

【样例 1 解释】

一种最优方案是令元素 3 的高度为 2，另外两个元素的高度为 1。三个元素的单次查询代价分别为 1, 2, 1。

【样例 2 输入】

```
1 10 2
2 1 1 1 1 1 1 1 1 1
```

【样例 2 输出】

```
1 24
```

【样例 3 输入】

```
1 1 200
2 1000000000
```

【样例 3 输出】

```
1 1000000000
```

【样例 4】

输入与输出见 sample 目录下的 *jump/jump4.in* 和 *jump/jump4.ans*。

【数据范围】

对于所有测试数据， $1 \leq N, K \leq 200$ ， $0 \leq x_i \leq 10^8$ 。

本题采用捆绑测试。

只有通过一个子任务中的全部测试，才能获得该子任务的分数。

子任务	分值	附加限制
1	10	$K = 1$
2	15	$K = 2$
3	15	$N \leq 15$ 且 $K \leq 3$
4	15	$K \geq N - 1$
5	45	无附加限制

各子任务之间没有依赖关系。

幸运评级 (rating)

【题目描述】

评级系统会把一个评级串与同长度的幸运串进行比较。两者足够相近时，这次评级会被记为幸运评级。

给定正整数 n 、整数序列 $a_1, a_2, \dots, a_n$ 和一个长度为 n 的大写英文字母串 S 。你需要统计不同评级串 T 的数量，使得：

- T 的长度为 n ；
- 对每个 $1 \leq i \leq n$ ， T_i 是前 a_i 个大写英文字母之一；
- T 与 S 的最长公共子序列长度至少为 $n - 2$ 。

一个字符串的子序列可以由原串删除若干字符得到，剩余字符的相对顺序不变，但不要求在原串中连续。两个字符串的最长公共子序列是同时为二者子序列的字符串中长度最大的一个。

每个满足条件的评级串 T 只计数一次。
答案对 998244353 取模。

【输入格式】

从 *rating.in* 读入数据。
第一行一个整数 n 。
第二行 n 个整数 $a_1, a_2, \dots, a_n$ 。
第三行一个长度为 n 的大写英文字母串 S 。

【输出格式】

输出到 *rating.out* 文件。
输出一个整数，表示满足条件的评级串 T 的数量对 998244353 取模后的结果。

【样例 1 输入】

```
1 1
2 3
3 Z
```

【样例 1 输出】

```
1 3
```

【样例 1 解释】

此时 $n - 2 = -1$ ，所以 A、B、C 三个可选评级串都满足条件。幸运串中的字符不必属于同一位置允许选择的字符范围。

【样例 2 输入】

```
1 5
2 1 2 1 2 1
3 ABABA
```

【样例 2 输出】

```
1 4
```

【样例 2 解释】

满足条件的评级串为 AAAAA、AAABA、ABAAA 和 ABABA。

【样例 3】

输入与输出见 sample 目录下的 *rating/rating3.in* 和 *rating/rating3.ans*。

【数据范围】

- 对于所有测试数据：
- $1 \leq n \leq 3 \times 10^5$ ；
 - $1 \leq a_i \leq 26$ ；
 - S 仅由大写英文字母 A 到 Z 组成。
- 本题采用捆绑测试。
只有通过一个子任务中的全部测试点，才能获得该子任务的分数。

子任务	分值	附加限制
1	20	$1 \leq n \leq 4$
2	60	$n \leq 10^4$
3	20	无额外限制

各子任务之间没有依赖关系。

打麻将 (mahjong)

【题目描述】

有 N 种牌，编号为 $1, 2, \dots, N$ ，每种牌都有无限张。初始手牌共有 M 张，第 j 张牌的编号为 a_j 。

随后不断摸牌。每次摸到的牌相互独立，并且等概率地取自 N 种牌之一；摸到的牌会加入手牌。

一个面子可以是以下两种形式之一：

- 刻子：三张编号相同的牌 (i, i, i) ，其中 $1 \leq i \leq N$ ；
- 顺子：三张编号连续的牌 $(i, i + 1, i + 2)$ ，其中 $1 \leq i \leq N - 2$ 。

同一张实体牌不能用于多个面子。若能从手牌中选出至少 K 个两两不共用牌的面子，就称当前手牌已经和牌。

设随机变量 T 表示从初始状态开始，直到第一次和牌为止所摸的牌数；使手牌第一次和牌的那次摸牌计入 T 。若初始手牌已经和牌，则 $T = 0$ 。求 $\mathbb{E}[T]$ 。

答案是一个有理数。若最简结果为 p/q ，输出

$$p \cdot q^{-1} \bmod 998244353,$$

其中 q^{-1} 表示 q 在模 998244353 意义下的乘法逆元。数据保证该逆元存在。

【输入格式】

从 *mahjong.in* 读入数据。

第一行三个整数 N, M, K 。

第二行包含 M 个整数 $a_1, a_2, \dots, a_M$ 。当 $M = 0$ 时，第二行可以为空行，也可以省略。

【输出格式】

输出到 *mahjong.out* 文件。

输出一个整数，表示期望摸牌张数对 998244353 取模后的结果。

【样例 1 输入】

```
1 3 4 1
2 1 1 3 3
```

【样例 1 输出】

```
1 1
```

【样例 1 解释】

无论第一张摸到哪一种牌，都可以组成一个面子。

【样例 2 输入】

```
1 3 3 1
2 1 2 2
```

【样例 2 输出】

```
1 332748119
```

【样例 2 解释】

第一张摸到 2 或 3 时立即和牌；第一张摸到 1 时还需再摸一张。因此期望为 $\frac{2}{3} \times 1 + \frac{1}{3} \times 2 = \frac{4}{3}$ 。

【样例 3 输入】

```
1 3 0 10
```

【样例 3 输出】

```
1 664057154
```

【样例 4】

输入与输出见 sample 目录下的 *mahjong/mahjong4.in* 和 *mahjong/mahjong4.ans*。

【数据范围】

对于所有测试数据：

- $1 \leq N \leq 100$ ；
- $1 \leq K \leq 20$ ；
- $0 \leq M \leq 100$ ；
- $1 \leq a_j \leq N$ 。

本题采用捆绑测试。

只有通过一个子任务中的全部测试点，才能获得该子任务的分数。

子任务	分值	附加限制
1	10	$N \leq 2$
2	20	$N \leq 3$
3	20	$K = 1$
4	50	无附加限制

各子任务之间没有依赖关系。

树上守望 (watch)

【题目描述】

一片林区由一棵有 n 个节点的带权树表示，节点编号为 1 到 n 。第 i 个节点设立守望点的费用为 c_i ，能够接受的最远守望距离为 d_i ；每条边都有一个正整数长度。

你需要选择若干节点设立守望点。对于每个节点 u ，必须至少存在一个被选中的节点 v ，使得它们之间的树上加权距离满足

$$\text{dist}(u, v) \leq d_u.$$

请最小化所有被选节点的费用之和。

【输入格式】

从 `watch.in` 读入数据。

本题有多组数据。

第一行一个整数 T ，表示数据组数。

对于每组数据：

- 第一行一个整数 n ；
- 第二行 n 个整数 $c_1, c_2, \dots, c_n$ ；
- 第三行 n 个整数 $d_1, d_2, \dots, d_n$ ；
- 接下来 $n - 1$ 行，每行三个整数 u, v, w ，表示节点 u 与节点 v 之间有一条长度为 w 的边。

输入保证每组数据给出的图是一棵树。

【输出格式】

输出到 `watch.out` 文件。

对于每组数据，输出一行一个整数，表示满足所有节点要求的最小总费用。

【样例 1 输入】

```
1 1
2 1
3 7
4 1
```

【样例 1 输出】

```
1 7
```

【样例 1 解释】

只有一个节点，因此必须在该节点设立守望点，费用为 7。

【样例 2 输入】

```
1 1
2 6
3 9 4 7 3 8 5
4 2 2 3 1 3 1
5 1 2 2
6 2 3 2
7 2 4 1
8 4 5 3
9 4 6 1
```

【样例 2 输出】

```
1 7
```

【样例 2 解释】

选择节点 2 和节点 4，总费用为 $4 + 3 = 7$ ，可以满足所有节点的距离要求。不存在费用更低的可行方案。

【样例 3】

输入与输出见 sample 目录下的 `watch/watch3.in` 和 `watch/watch3.ans`。

【数据范围】

- 对于所有测试数据：
- $1 \leq T \leq 10$ ；
 - $1 \leq n \leq 1000$ ；
 - $1 \leq c_i, d_i, w \leq 10^9$ 。
- 本题采用捆绑测试。
只有通过一个子任务中的全部测试点，才能获得该子任务的分数。

子任务	分值	附加限制
1	15	$n \leq 20$
2	15	树是一条链
3	15	除节点 1 外，每个节点都与节点 1 直接相连
4	15	对所有节点与边均有 $d_i = 1, w = 1$
5	20	所有边权均为 1，且所有 c_i 相等
6	20	无附加限制

同一输入文件中的每组数据均满足该文件所属子任务的附加限制。各子任务之间没有依赖关系。